Programmation procédurale

10 exercices de rappel en C

Syntaxe, conditions, boucles, tableaux et fonctions

Cours	Techniques de programmation procédurale
Professeur	Romain Chariot
Année académique	2026 - 2027
Établissement	Institut de Technologie - Enseignement pour adultes de la Ville de Liège

But de la séance

Ces exercices servent de remise à niveau avant d’aborder des notions plus avancées du langage C. Ils permettent de retravailler les entrées/sorties, les conditions, les boucles, les tableaux et la décomposition en fonctions.

Institut de Technologie - Quai du Condroz, 15 - 4020 Liège

Avant de commencer

Consignes générales

- Chaque exercice doit être développé en langage C et exécuté en console.
- Privilégiez un code simple, lisible et correctement découpé en fonctions.
- Donnez des noms explicites aux variables et aux fonctions.
- Testez les cas normaux mais également les entrées incorrectes ou limites.
- Les exemples d'exécution sont indicatifs : votre interface peut être différente si elle reste claire.
- Vous devez être capable d'expliquer votre solution et les choix réalisés.

Bibliothèques rencontrées

Bibliothèque	Utilité principale
`<stdio.h>`	printf, scanf : entrées et sorties console
`<stdbool.h>`	type bool, valeurs true et false
`<stdlib.h>`	rand, srand et fonctions utilitaires
`<time.h>`	initialisation du générateur pseudo-aléatoire
`<math.h>`	fonctions mathématiques telles que sqrt

Attention

Pour les exercices qui utilisent des nombres aléatoires, initialisez le générateur une seule fois dans le programme avec srand(time(NULL)). Pour Pythagore, sqrt() nécessite math.h.

Parcours des exercices

#	Exercice	Notions principales
01	Calculatrice avec menu	Fonctions, switch, do...while, calculs
02	Pair, impair et multiples	Conditions, booléens, modulo
03	Jeu de devinettes	Boucles, conditions, aléatoire
04	Convertisseur d’unités	Fonctions, calculs, menu
05	Analyse d’une série de notes	Tableaux, boucles, statistiques simples
06	Tables de multiplication	Boucles for, boucles imbriquées
07	Compte bancaire	État, conditions, menu
08	Recherche dans un tableau	Parcours, recherche linéaire
09	Pierre - Papier - Ciseaux	Aléatoire, logique conditionnelle
10	Gestionnaire de tâches	Tableaux de booléens, calcul de progression

Organisation conseillée

- Commencez par les exercices 1 à 4 pour remettre en place les automatismes.
- Poursuivez avec les exercices 5 à 8 pour retravailler les tableaux et les parcours.
- Terminez avec les exercices 9 et 10, qui combinent plusieurs notions.

Exercice 1

Calculatrice avec menu

Objectif

Créer une petite calculatrice organisée autour d’un menu et de fonctions dédiées à chaque opération.

Bibliothèques utiles

```
<stdio.h>                                   <math.h>
```

Travail demandé

- Afficher un sous-menu permettant de choisir une opération : addition, soustraction, multiplication, division ou calcul de l’hypoténuse.
- Demander les deux valeurs nécessaires au calcul.
- Créer une fonction distincte pour chaque opération.
- Empêcher une division par zéro.
- Permettre à l’utilisateur de revenir au menu principal.

Exemple d’exécution

```
=== CALCULATRICE ===
1. Addition
2. Soustraction
3. Multiplication
4. Division
5. Pythagore
0. Retour

Votre choix : 1
Premier nombre : 12
Deuxieme nombre : 8
Resultat : 20.00
```

Conseils

- Un switch est adapté à la gestion du sous-menu.
- Pour Pythagore : c = sqrt(a*a + b*b).
- Utilisez float ou double si vous souhaitez conserver les décimales.

Exercice 2

Pair, impair et multiples

Objectif

Analyser un entier à l’aide de conditions simples et de l’opérateur modulo.

Bibliothèques utiles

```
<stdio.h>                                   <stdbool.h>
```

Travail demandé

- Demander un entier à l’utilisateur.
- Indiquer si ce nombre est pair ou impair.
- Indiquer s’il est multiple de 3, de 5 et de 10.
- Créer au minimum une fonction permettant de tester si un nombre est multiple d’un autre.

Exemple d’exécution

```
=== ANALYSE D'UN NOMBRE ===
Entrez un nombre : 30

30 est pair.
30 est un multiple de 3.
30 est un multiple de 5.
30 est un multiple de 10.
```

Conseils

- L’opérateur % donne le reste d’une division entière.
- Si nombre % diviseur vaut 0, le nombre est multiple du diviseur.
- Une fonction de test peut retourner un bool.

Exercice 3

Jeu de devinettes

Objectif

Créer un jeu simple utilisant une boucle, des conditions et un nombre pseudo-aléatoire.

Bibliothèques utiles

```
<stdio.h>                    <stdlib.h>                    <time.h>
```

Travail demandé

- Générer un nombre secret compris entre 1 et 100.
- Demander des propositions jusqu'à ce que le nombre soit trouvé.
- Après chaque tentative, afficher Trop petit, Trop grand ou Bravo.
- Compter et afficher le nombre de tentatives.
- Bonus : permettre de rejouer une nouvelle partie.

Exemple d'exécution

```
=== DEVINETTES ===
Je pense a un nombre entre 1 et 100.

Votre proposition : 50
Trop petit !
Votre proposition : 75
Trop grand !
Votre proposition : 68
Bravo !
Nombre de tentatives : 3
```

Conseils

- rand() % 100 + 1 permet d'obtenir une valeur de 1 à 100.
- Initialisez le générateur pseudo-aléatoire avec srand(time(NULL)).
- Le nombre de tentatives est un simple compteur.

Exercice 4

Convertisseur d’unités

Objectif

Construire un menu de conversions et isoler chaque formule dans une fonction.

Bibliothèques utiles

```
<stdio.h>
```

Travail demandé

- Proposer au minimum : Celsius vers Fahrenheit, Fahrenheit vers Celsius, kilomètres vers miles et mètres vers pieds.
- Demander la valeur à convertir.
- Créer une fonction pour chaque conversion.
- Afficher le résultat avec deux chiffres après la virgule.
- Permettre de réaliser plusieurs conversions sans relancer le programme.

Exemple d’exécution

```
=== CONVERTISSEUR ===
1. Celsius -> Fahrenheit
2. Fahrenheit -> Celsius
3. Kilometres -> Miles
4. Metres -> Pieds
0. Retour

Votre choix : 1
Valeur : 20
20.00 C = 68.00 F
```

Conseils

- Gardez les formules simples et lisibles.
- Utilisez %.2f pour afficher deux décimales.
- Une fonction de conversion reçoit une valeur et retourne le résultat.

Exercice 5

Analyse d’une série de notes

Objectif

Parcourir un tableau pour calculer plusieurs indicateurs simples.

Bibliothèques utiles

```
<stdio.h>
```

Travail demandé

- Demander cinq notes et les stocker dans un tableau.
- Calculer la moyenne des notes.
- Déterminer la note minimale et la note maximale.
- Compter le nombre de notes supérieures ou égales à 10/20.
- Afficher un résumé clair des résultats.

Exemple d’exécution

```
=== ANALYSE DES NOTES ===
Note 1 : 12
Note 2 : 17
Note 3 : 8
Note 4 : 14
Note 5 : 16

Moyenne : 13.40
Minimum : 8
Maximum : 17
Reussites : 4 / 5
```

Conseils

- Initialisez minimum et maximum avec la première valeur du tableau.
- Parcourez ensuite l’ensemble du tableau avec une boucle.
- Attention à la division entière lors du calcul de la moyenne.

Exercice 6

Tables de multiplication

Objectif

Utiliser les boucles for pour générer automatiquement des séries de calculs.

Bibliothèques utiles

```
<stdio.h>
```

Travail demandé

- Demander un entier à l'utilisateur.
- Afficher sa table de multiplication de 1 à 10.
- Bonus : ajouter une option permettant d'afficher toutes les tables de 1 à 10.
- Pour le bonus, utiliser des boucles imbriquées.

Exemple d'exécution

```
=== TABLE DE MULTIPLICATION ===
Nombre : 7

7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
...
7 x 10 = 70
```

Conseils

- La boucle for est idéale lorsque le nombre d'itérations est connu.
- Une boucle peut être placée à l'intérieur d'une autre boucle.
- Évitez d'écrire dix printf différents.

Exercice 7

Gestion d’un compte bancaire

Objectif

Manipuler une valeur qui évolue au fil des actions de l’utilisateur.

Bibliothèques utiles

```
<stdio.h>
```

Travail demandé

- Initialiser un solde de départ.
- Créer un menu permettant d’effectuer un dépôt, un retrait ou d’afficher le solde.
- Refuser un retrait supérieur au solde disponible.
- Refuser les montants négatifs ou nuls.
- Le solde doit conserver sa nouvelle valeur tant que le programme reste ouvert.

Exemple d’exécution

```
=== COMPTE BANCAIRE ===
Solde : 500.00 EUR

1. Depot
2. Retrait
3. Afficher le solde
0. Retour

Votre choix : 2
Montant : 100
Nouveau solde : 400.00 EUR
```

Conseils

- La variable solde représente l’état courant du compte.
- Validez toujours le montant avant de modifier le solde.
- Pensez au cas où le retrait demandé est trop important.

Exercice 8

Recherche dans un tableau

Objectif

Mettre en place une recherche linéaire dans une collection de valeurs.

Bibliothèques utiles

```
<stdio.h>                                <stdbool.h>
```

Travail demandé

- Utiliser un tableau contenant plusieurs nombres.
- Demander à l'utilisateur la valeur recherchée.
- Parcourir le tableau afin de vérifier si cette valeur est présente.
- Afficher sa position si elle est trouvée.
- Afficher un message adapté si elle est absente.

Exemple d'exécution

```
=== RECHERCHE ===
Valeurs : 12 45 7 23 56 18 9

Nombre recherche : 23
23 a ete trouve a la position 4.
```

Conseils

- Commencez par une recherche simple de gauche à droite.
- Un bool peut mémoriser si une valeur a été trouvée.
- Décidez si vous affichez l'indice du tableau ou une position compréhensible par l'utilisateur.

Exercice 9

Pierre - Papier - Ciseaux

Objectif

Combiner génération aléatoire, conditions et menu dans un petit jeu.

Bibliothèques utiles

```
<stdio.h>                    <stdlib.h>                    <time.h>
```

Travail demandé

- Permettre au joueur de choisir Pierre, Papier ou Ciseaux.
- Générer aléatoirement le choix de l'ordinateur.
- Comparer les deux choix et annoncer victoire, défaite ou égalité.
- Bonus : jouer plusieurs manches et conserver le score.

Exemple d'exécution

```
=== PIERRE - PAPIER - CISEAUX ===
1. Pierre
2. Papier
3. Ciseaux

Votre choix : 1
Ordinateur : Ciseaux
Vous gagnez !
```

Conseils

- Vous pouvez représenter Pierre, Papier et Ciseaux par les valeurs 1, 2 et 3.
- Commencez par traiter l'égalité, puis les trois situations de victoire.
- Toutes les autres combinaisons correspondent alors à une défaite.

Exercice 10

Mini gestionnaire de tâches

Objectif

Utiliser un tableau de booléens et calculer un état d'avancement.

Bibliothèques utiles

```
<stdio.h>                                   <stdbool.h>
```

Travail demandé

- Créer une liste de cinq tâches.
- Associer à chaque tâche un état : terminée ou non terminée.
- Afficher la liste avec un indicateur visuel [ ] ou [X].
- Permettre à l'utilisateur de marquer une tâche comme terminée.
- Calculer et afficher le nombre de tâches terminées ainsi que le pourcentage de progression.

Exemple d'exécution

```
=== GESTIONNAIRE DE TACHES ===
1. [ ] Installer CLion
2. [X] Creer le repository Git
3. [ ] Terminer le labo
4. [ ] Commit et Push
5. [ ] Relire le cours

Taches terminees : 1 / 5
Progression : 20.00 %
```

Conseils

- Un tableau de booléens suffit pour mémoriser l'état des tâches.
- Pour calculer le pourcentage : terminees / total * 100.
- Attention à convertir une valeur en float pour éviter la division entière.